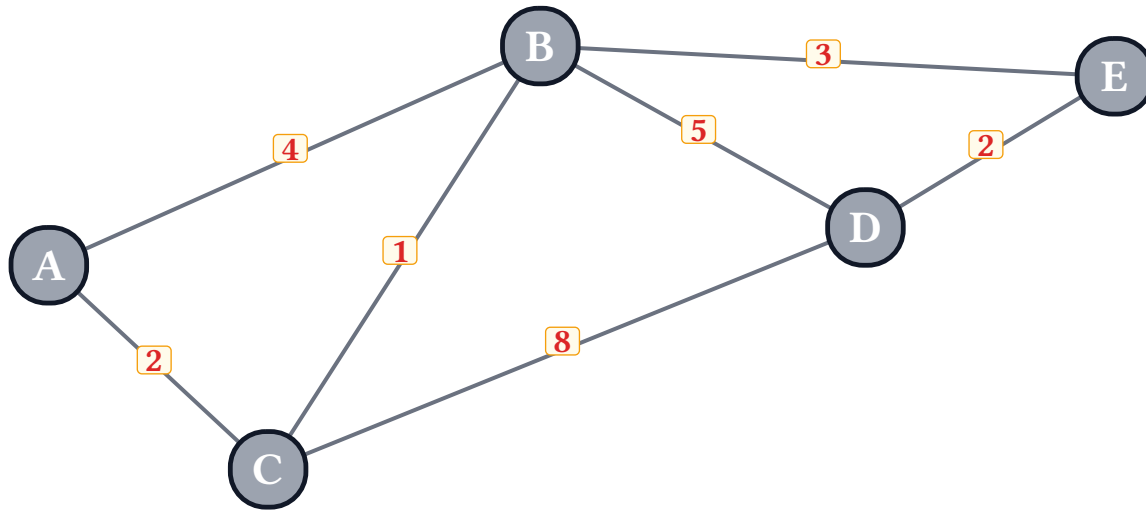


Shortest Path on a Weighted Graph – Dijkstra's Algorithm

Goal: Find the minimum-cost path from source **A** to every other vertex.



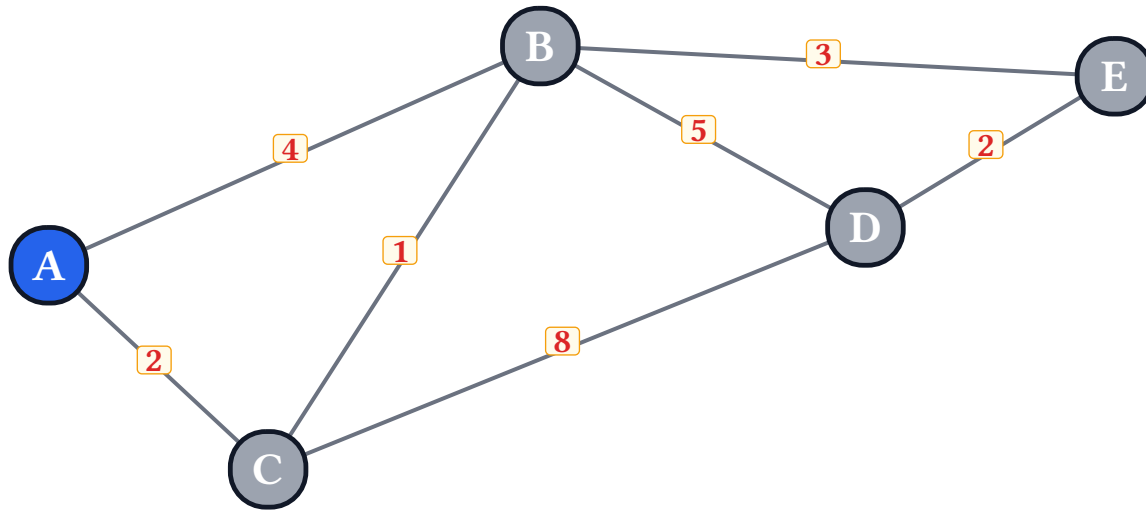
Undirected weighted graph $G = (V, E)$

Edge	Weight
A – B	4
A – C	2
C – B	1
B – D	5
C – D	8
B – E	3
D – E	2

Source node: A

Dijkstra greedily expands the nearest unvisited vertex and **relaxes** its neighbors.

Step 0 – Initialize



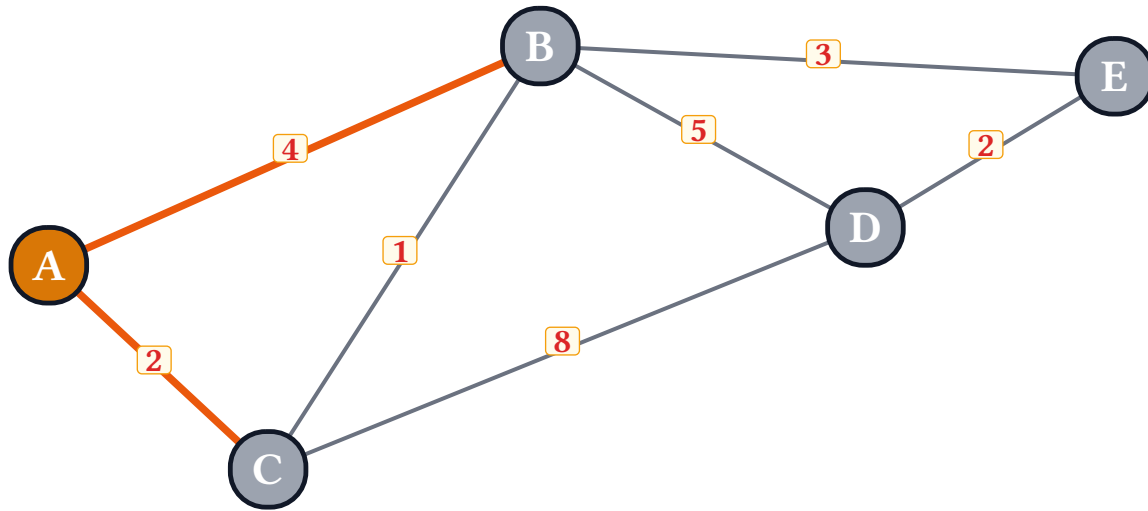
V	dist	via	status
A	0	—	queued
B	∞	—	unknown
C	∞	—	unknown
D	∞	—	unknown
E	∞	—	unknown

Queue: { A : 0 }

- $\text{dist}[A] = 0$ (source)
- All other $\text{dist} = \infty$
- Insert A into min-priority queue

Set $\text{dist}[\text{source}] = 0$, all others $= \infty$. Add source to the priority queue.

Step 1 — Visit A (dist = 0) · Relax neighbors B and C



V	dist	via	status
A	0 ✓	—	done
B	4	A	queued
C	2	A	queued
D	∞	—	unknown
E	∞	—	unknown

Queue: { C : 2, B : 4 }

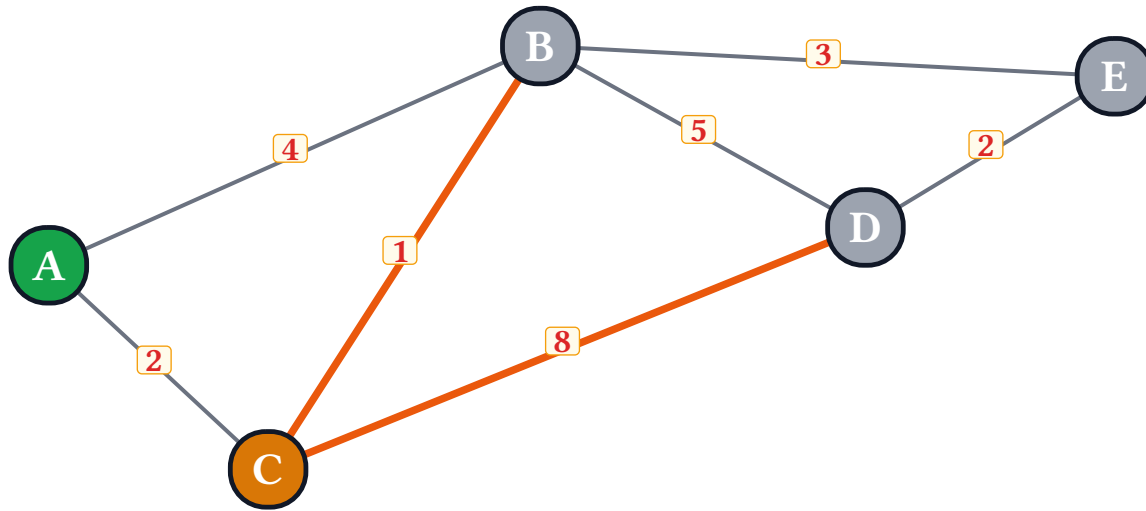
Pop A (smallest dist). Relax:

- A—B: $0 + 4 = 4$ ↓ new
- A—C: $0 + 2 = 2$ ↓ new

Mark A done. C is now closest.

Both neighbors of A are discovered. C gets the smaller distance so it leads the queue.

Step 2 — Visit C (dist = 2) · Relax neighbors B and D



V	dist	via	status
A	0 ✓	—	done
B	3	C	updated!
C	2 ✓	A	done
D	10	C	queued
E	∞	—	unknown

Queue: { B : 3, D : 10 }

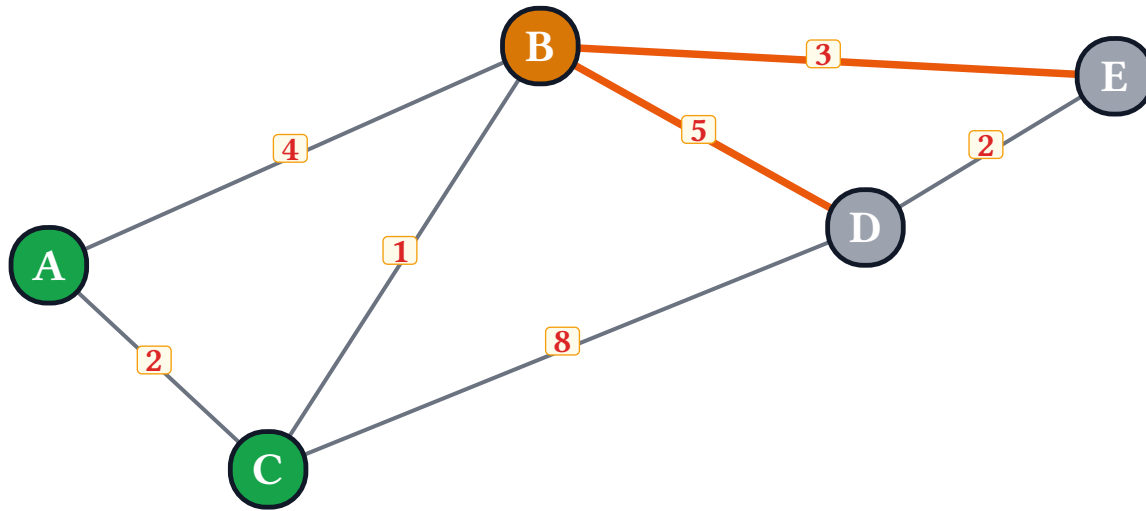
Pop C (smallest dist = 2). Relax:

- C—B: $2 + 1 = 3 < 4$ ↓ improved!
- C—D: $2 + 8 = 10$ ↓ new

Mark C done. B's path improved via C.

Key relaxation: B's distance drops from 4 to 3 by going through C. $\text{dist}[B] = 3$ beats the direct A—B edge (4).

Step 3 — Visit B (dist = 3) · Relax neighbors D and E



V	dist	via	status
A	0 ✓	—	done
B	3 ✓	C	done
C	2 ✓	A	done
D	8	B	updated!
E	6	B	queued

Queue: { E : 6, D : 8 }

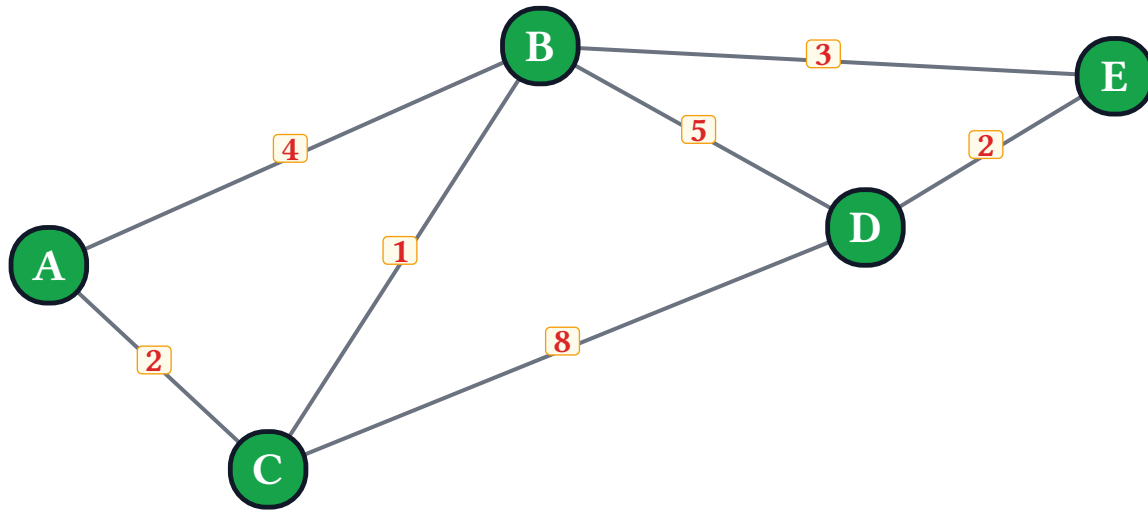
Pop **B** (smallest dist = 3). Relax:

- B—D: $3 + 5 = 8 < 10$ ↓ improved!
- B—E: $3 + 3 = 6$ ↓ new

Mark B done. D improved again; E discovered.

D's distance improves a second time ($10 \rightarrow 8$). E is seen for the first time at cost 6.

Steps 4 & 5 — Visit E (dist = 6), then D (dist = 8) — Finalize



V	dist	via	status
A	0 ✓	—	done
B	3 ✓	C	done
C	2 ✓	A	done
D	8 ✓	B	done
E	6 ✓	B	done

Step 4 — Pop E (dist = 6):

- D: $6 + 2 = 8$ = same → no update
- B: already done → skip

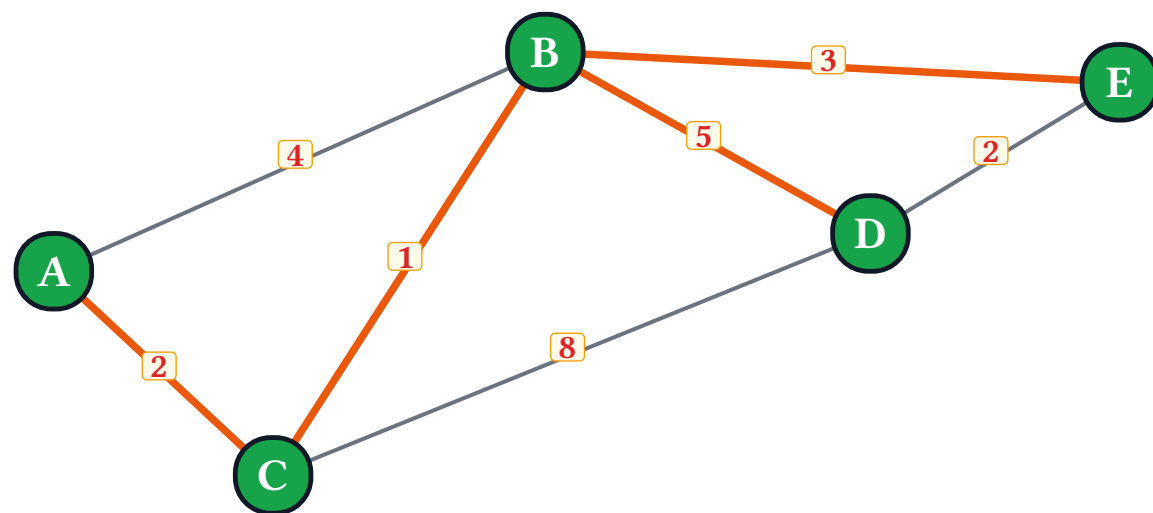
Step 5 — Pop D (dist = 8):

- E: already done → skip

Queue empty — **all vertices finalized!**

No further improvements are possible. Every vertex has its shortest distance from A confirmed.

Final Result – Shortest Path Tree from A



— Highlighted edges form the Shortest Path Tree

To	Cost	Path
A	0	A
C	2	A → C
B	3	A → C → B
E	6	A → C → B → E
D	8	A → C → B → D

Key insight: The direct A—B edge (cost 4) is NOT in the shortest path tree. The detour A→C→B costs only 3.